

บทที่ 2 การบูรณาการข้อมูล

วัตถุประสงค์การเรียนรู้

1. อธิบายความหมายและความสำคัญของการบูรณาการข้อมูล รวมถึงความท้าทายหลัก ได้แก่ Structural Heterogeneity, Semantic Heterogeneity และ Entity Resolution
2. ระบุและอธิบายปัญหาที่พบบ่อยในการบูรณาการข้อมูล ได้แก่ Entity Resolution, Missing Values และ Inconsistent Values พร้อมแนวทางแก้ไข
3. อธิบายมิติของคุณภาพข้อมูล ได้แก่ Accuracy, Completeness, Timeliness และ Consistency ตามหลักการ “Garbage In, Garbage Out”
4. จำแนกประเภทการรวมข้อมูลเชิงทฤษฎี ได้แก่ Vertical Concatenation, Join Operations (Inner, Outer, Left, Right, Cross Join) และ Index-Based Join
5. อธิบายกระบวนการเตรียมข้อมูลก่อนการบูรณาการ ได้แก่ Aggregation, Sampling และ Variable Transformation รวมถึง Normalization และ Standardization
6. ใช้ฟังก์ชัน `concat()` ของ Pandas เพื่อต่อข้อมูลทั้งแนวนอน (axis=0) และแนวคอลัมน์ (axis=1)
7. ใช้ฟังก์ชัน `merge()` เพื่อรวมข้อมูลโดยอ้างอิง Key Column รองรับรูปแบบ Inner, Outer, Left และ Right Join
8. ใช้ฟังก์ชัน `join()` เพื่อเชื่อมต่อ DataFrame โดยใช้ Index และประยุกต์ใช้การบูรณาการข้อมูลในกรณีศึกษาเงินเดือนนักวิทยาศาสตร์ข้อมูล

2.1 บทนำ

การบูรณาการข้อมูล (Data Integration) คือกระบวนการรวบรวมข้อมูลจากแหล่งต่างๆ เข้ามาสู่มุมมองเดียวที่เปลี่ยนหนึ่งเดียวกัน กระบวนการนี้เกี่ยวข้องกับการรวมข้อมูลจากประเภทข้อมูล โครงสร้าง และรูปแบบที่แตกต่างกันเพื่อสร้างชุดข้อมูลเดียวที่สามารถนำไปใช้สำหรับการวิเคราะห์และการรายงาน ขั้นตอนนี้มีความสำคัญเนื่องจากช่วยให้สามารถวิเคราะห์ข้อมูลจากหลายแหล่ง ซึ่งสามารถให้ภาพที่สมบูรณ์และถูกต้องมากขึ้นของข้อมูลที่กำลังวิเคราะห์

Python มีแพ็คเกจที่ใช้กันทั่วไปสำหรับการบูรณาการข้อมูล ได้แก่:

9. Pandas: ไลบรารีที่ทรงพลังสำหรับการจัดการและวิเคราะห์ข้อมูล มีโครงสร้างข้อมูลอย่าง DataFrame และ Series ที่ช่วยให้รวม กรอง แปลง และจัดรูปร่างข้อมูลได้
10. NumPy: ไลบรารีที่ทรงพลังสำหรับการคำนวณอาร์เรย์ มีออบเจกต์อาร์เรย์หลายมิติที่มีประสิทธิภาพสูง และเครื่องมือสำหรับทำงานกับอาร์เรย์เหล่านั้น

2.2 แนวคิดและทฤษฎีที่เกี่ยวข้อง

2.2.1 ความหมายและขอบเขตของการบูรณาการข้อมูล

การบูรณาการข้อมูล (Data Integration) เป็นกระบวนการสำคัญในกระบวนการทำเหมืองข้อมูล (Data Mining) ที่มุ่งรวบรวมข้อมูลจากแหล่งต่างๆ ที่มีความหลากหลายให้กลายเป็นชุดข้อมูลที่สอดคล้องและสามารถนำไปวิเคราะห์ได้อย่างมีประสิทธิภาพ Tan, Steinbach และ Kumar (2006) ได้ให้คำจำกัดความว่า ชุดข้อมูลสามารถมองได้เป็นกลุ่มของ data objects ซึ่งแต่ละ object ถูกอธิบายด้วย attributes ที่สะท้อนคุณลักษณะพื้นฐานของสิ่งที่ต้องการศึกษา

ดังนั้นการบูรณาการข้อมูลจึงต้องคำนึงถึงการจัดการทั้งในระดับโครงสร้าง ความหมาย และคุณภาพของข้อมูลจากแต่ละแหล่งด้วย

ในทางปฏิบัติ กระบวนการบูรณาการข้อมูลครอบคลุมความท้าทายหลัก 3 ประการ ได้แก่ (1) ความแตกต่างในโครงสร้างของข้อมูล (Structural Heterogeneity) ซึ่งเกิดขึ้นเมื่อแหล่งข้อมูลต่างๆ จัดเก็บข้อมูลในรูปแบบและโครงสร้างที่ต่างกัน (2) ความแตกต่างในความหมาย (Semantic Heterogeneity) ซึ่งเกิดขึ้นเมื่อข้อมูลขึ้นเดียวกันมีการตีความหรือการนิยามที่แตกต่างกันในแต่ละแหล่ง และ (3) ปัญหาเอนทิตีซ้ำซ้อน (Entity Resolution) ซึ่งเกิดขึ้นเมื่อข้อมูลจากแหล่งต่างๆ อ้างถึงสิ่งเดียวกันแต่มีการระบุชื่อหรือคีย์ที่แตกต่างกัน

2.2.2 ปัญหาในการบูรณาการข้อมูล

ปัญหาหลักที่พบบ่อยในการบูรณาการข้อมูล มีดังต่อไปนี้

1) ปัญหาเอนทิตีซ้ำซ้อน (Entity Resolution / Deduplication)

ชุดข้อมูลจากแหล่งต่างๆ อาจมีข้อมูลที่อ้างถึงสิ่งเดียวกันซ้ำซ้อนกันหลายครั้ง โดยใช้ชื่อหรือรหัสที่แตกต่างกัน (Tan et al., 2006) ยกตัวอย่างเช่น บุคคลคนเดียวกันอาจถูกบันทึกไว้ในหลายฐานข้อมูลด้วยที่อยู่ที่แตกต่างกัน หรือสินค้าชนิดเดียวกันอาจมีรหัสสินค้าแตกต่างกันในระบบต่างๆ กระบวนการที่ใช้แก้ไขปัญหานี้เรียกว่า Deduplication โดยต้องอาศัยการตรวจสอบและเปรียบเทียบค่าของ attributes หลายๆ ตัวเพื่อระบุว่าข้อมูลสองชิ้นหมายถึงสิ่งเดียวกันหรือไม่

2) ปัญหาค่าสูญหายไป (Missing Values)

ข้อมูลที่รวบรวมจากหลายแหล่งมักมีค่าสูญหาย ซึ่งเกิดขึ้นได้จากความผิดพลาดในการเก็บข้อมูล ความแตกต่างของโครงสร้าง หรือขอบเขตการเก็บข้อมูลที่แตกต่างกัน แนวทางในการจัดการค่าสูญหายมีหลายวิธี เช่น การลบ record ที่มีค่าขาดหาย (Elimination)

การประมาณค่าโดยใช้ค่าเฉลี่ยหรือค่ากลางอื่นๆ (Imputation) หรือการใช้อัลกอริทึมที่สามารถจัดการค่าสูญหายได้โดยตรง (Tan et al., 2006)

3) ปัญหาค่าที่ขัดแย้งกัน (Inconsistent Values)

เมื่อรวมข้อมูลจากแหล่งต่างๆ ค่าของ attribute เดียวกันอาจขัดแย้งกัน เช่น ข้อมูลที่อยู่ ซึ่งมีรหัสไปรษณีย์ที่ไม่ตรงกับจังหวัดที่ระบุไว้ หรือข้อมูลอุณหภูมิที่เก็บจากสองแหล่งที่มีการสอบเทียบเครื่องมือวัดที่แตกต่างกัน กระบวนการตรวจสอบและแก้ไขความไม่สอดคล้องกันนี้เรียกว่า Data Cleaning ซึ่งเป็นขั้นตอนสำคัญในกระบวนการบูรณาการข้อมูล

2.2.3 คุณภาพของข้อมูล (Data Quality)

คุณภาพของข้อมูลเป็นองค์ประกอบที่ขาดไม่ได้ในกระบวนการบูรณาการข้อมูล เนื่องจากการนำข้อมูลคุณภาพต่ำเข้าสู่กระบวนการวิเคราะห์จะส่งผลให้ผลลัพธ์ที่ได้ขาดความน่าเชื่อถือ ตามหลักการ “Garbage In, Garbage Out” Tan et al. (2006) ได้แบ่งมิติของคุณภาพข้อมูลออกเป็นองค์ประกอบสำคัญดังนี้

ความแม่นยำ (Accuracy) หมายถึงความใกล้เคียงของค่าที่วัดได้กับค่าจริง (true value) ซึ่งขึ้นอยู่กับทั้งความเที่ยงตรง (Precision) ซึ่งวัดจากความสม่ำเสมอของการวัดซ้ำ และค่าเบี่ยงเบน (Bias) ซึ่งวัดจากความแตกต่างเป็นระบบระหว่างค่าเฉลี่ยที่วัดได้กับค่าจริง

ความสมบูรณ์ (Completeness) หมายถึงการที่ข้อมูลมีครบถ้วนตามที่ต้องการสำหรับการวิเคราะห์ ปัญหาความไม่สมบูรณ์เกิดขึ้นได้จากหลายสาเหตุ เช่น ข้อผิดพลาดในการบันทึก ข้อจำกัดของระบบจัดเก็บ หรือแหล่งข้อมูลมีขอบเขตการเก็บข้อมูลที่แตกต่างกัน

ความทันสมัย (Timeliness) ข้อมูลบางประเภทมีอายุการใช้งานจำกัด เช่น ข้อมูลพฤติกรรมมนุษย์บนโซเชียลมีเดีย หรือข้อมูลการท่องเที่ยว ซึ่งสะท้อนความเป็นจริงเพียงช่วงเวลาหนึ่งเท่านั้น หากข้อมูลล้าสมัย โมเดลที่สร้างจากข้อมูลนั้นก็จะสูญเสียความถูกต้องตามไปด้วย

ค ว า ม ส อ ด ค ล ั อ ง (C o n s i s t e n c y)

หมายถึงการที่ค่าของข้อมูลไม่ขัดแย้งกันทั้งภายในชุดข้อมูลเดียวกันและระหว่างแหล่งข้อมูลต่างๆ ความไม่สอดคล้องกันมักเกิดจากมาตรฐานหรือคำนิยามที่ต่างกัน เช่น หน่วยวัดที่แตกต่าง หรือรูปแบบวันที่ที่ไม่เป็นมาตรฐาน

2.2.4 ประเภทของการรวมข้อมูลเชิงทฤษฎี

จากมุมมองเชิงทฤษฎี การรวมข้อมูลตารางสัมพันธ์ (Relational Data) สามารถอธิบายได้ผ่านหลักการของ Relational Algebra ซึ่งเป็นรากฐานทางคณิตศาสตร์ของระบบฐานข้อมูล โดยแบ่งประเภทการรวมได้ดังนี้

การต'อ แนวนต'ัง (V e r t i c a l C o n c a t e n a t i o n / U n i o n)

คือการนำชุดข้อมูลหลายชุดที่มีโครงสร้างเดียวกัน (schema เหมือนกัน) มา เรียง ต'อ กั น ใน แนวน แถว โดยแต่ละชุดข้อมูลแทนกลุ่มตัวอย่างที่แตกต่างกันจากประชากรเดียวกัน ในทางคณิตศาสตร์ตรงกับ Set Union ใน Relational Algebra

การรวมตามคีย์ร'วม (J o i n O p e r a t i o n s)

คือการนำชุดข้อมูลสองชุดมารวมกันโดยอ้างอิงคอลัมน์ที่มีค่าร่วมกัน (Key Column) ซึ่งมีรูปแบบย่อยหลายประเภท ได้แก่ Inner Join ที่คืนเฉพาะแถวที่มี Key ตรงกันในทั้งสองตาราง, Outer Join ที่คืนทุกแถวจากทั้งสองตารางและเติม NULL สำหรับค่าที่ไม่มีข้อมูลตรงกัน, Left/Right Join ที่คืนทุกแถวจากตารางข้างใดข้างหนึ่งเป็นหลัก และ Cross Join ที่สร้าง Cartesian Product ของทุกแถวจากทั้งสองตาราง การเลือกประเภท Join ที่เหมาะสมขึ้นอยู่กับลักษณะของข้อมูลและวัตถุประสงค์ของการวิเคราะห์

การรวมตามดัชนี (Index-Based Join) เป็นรูปแบบพิเศษของการ Join ที่ใช้ Index ของข้อมูล (แทนที่จะเป็นค่าใน Column) เป็นตัวกำหนดการจับคู่แถว มีประโยชน์เมื่อชุดข้อมูลทั้งสองมีการจัดเรียง Index ที่สัมพันธ์กันอยู่แล้ว

2.2.5 การเตรียมข้อมูลก่อนการบูรณาการ (Data Preprocessing)

ก่อนดำเนินการบูรณาการข้อมูล จำเป็นต้องผ่านกระบวนการเตรียมข้อมูล (Data Preprocessing) ซึ่ง Tan et al. (2006) ได้แบ่งออกเป็นขั้นตอนหลักที่มีความสัมพันธ์กันดังนี้

การรวมข้อมูล (Aggregation) คือการนำ data objects ตั้งแต่สองรายการขึ้นไปมารวมเป็นรายการเดียว เช่น การรวมข้อมูลรายวันเป็นรายเดือน หรือการรวมข้อมูลการขายรายสาขาเป็นยอดรวมระดับบริษัท ซึ่งช่วยลดปริมาณข้อมูลและลดความผันผวนของสัญญาณรบกวน (noise)

การสุ่มตัวอย่าง (Sampling) เป็นเทคนิคสำคัญในการลดขนาดข้อมูลโดยเลือกชุดย่อยของข้อมูลที่เป็นตัวแทนของประชากรทั้งหมด เพื่อให้สามารถวิเคราะห์ได้อย่างมีประสิทธิภาพ โดยต้องระวังปัญหา Sampling Bias ซึ่งเกิดขึ้นเมื่อตัวอย่างไม่ได้สะท้อนสัดส่วนของกลุ่มต่างๆ ในประชากรอย่างถูกต้อง

การแปลงตัวแปร (Variable Transformation) เป็นกระบวนการปรับเปลี่ยนค่าของตัวแปรเพื่อให้ข้อมูลจากแหล่งต่างๆ อยู่ในหน่วยหรือสเกลเดียวกัน เช่น การ Normalization เพื่อให้ค่าอยู่ในช่วง [0, 1] หรือการ Standardization เพื่อให้ค่าเฉลี่ยเป็น 0 และส่วนเบี่ยงเบนมาตรฐานเป็น 1 ซึ่งมีความสำคัญอย่างมากก่อนการนำข้อมูลจากหลายแหล่งมารวมกัน เนื่องจากข้อมูลที่มีสเกลต่างกันมากอาจทำให้อัลกอริทึมบางประเภทให้น้ำหนักกับตัวแปรบางตัวมากเกินไป

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

$$z = \frac{x - \mu}{\sigma}$$

ตารางที่ 2.1 สรุปความสัมพันธ์ระหว่างปัญหาในการบูรณาการข้อมูลและแนวทางแก้ไข

ปัญหา	สาเหตุ	แนวทางแก้ไข
Entity Resolution	ข้อมูลซ้ำจากหลายแหล่ง ใช้ชื่อ/รหัสต่างกัน	Deduplication, Record Linkage

Missing Values	ขอบเขตการเก็บข้อมูลต่างกัน ความผิดพลาดในการบันทึก	Elimination, Imputation, ใช้อัลกอริทึมที่ทนต่อค่า missing
Inconsistent Values	มาตรฐานต่างกัน หน่วยวัดหรือคำนิยามต่างกัน	Data Cleaning, Normalization, Standardization
Structural Heterogeneity	รูปแบบและโครงสร้างข้อมูลต่างกัน	Schema Mapping, ETL Process

ที่มา: ปรับปรุงจาก Tan, Steinbach, and Kumar (2006), *Introduction to Data Mining*, Chapter 2

2.3 การบูรณาการข้อมูล

วัตถุประสงค์: เก็บรวบรวมข้อมูลจากไฟล์ต่างๆ ฐานข้อมูล SQLite และเว็บไซต์สำหรับลูกค้า ขั้นตอนในการดำเนินการตามคำขอมิดังนี้:

11. ท ำ ค ว า ม เชื่ อ ใจ ค ว า ม ต อ ง ก า ร :
นัดประชุมกับลูกค้าเพื่อรวบรวมความต้องการโดยละเอียด กำหนดไฟล์เฉพาะ
ฐานข้อมูล SQLite และเว็บไซต์ที่ลูกค้าต้องการเก็บข้อมูล
ชี้แจงรูปแบบข้อมูลที่ต้องการ เกณฑ์การดึงข้อมูล
และความต้องการในการประมวลผลข้อมูลเฉพาะ
12. การเก็บรวบรวมข้อมูลจากไฟล์: ระบุรูปแบบไฟล์ (เช่น CSV, HTML, TXT, XLSX,
JSON) และตำแหน่งที่ตั้ง ใช้ไลบรารี Python ที่เหมาะสม (เช่น Pandas)
เพื่ออ่านและดึงข้อมูลจากแต่ละรูปแบบไฟล์
จัดเก็บข้อมูลที่ดึงมาในรูปแบบที่รวมเป็นหนึ่ง (เช่น DataFrame)
13. การเก็บรวบรวมข้อมูลจากฐานข้อมูล SQLite: ใช้ไลบรารี Python (เช่น sqlite3)
เพื่อสร้างการเชื่อมต่อ ดำเนินการ SQL Query
เพื่อดึงข้อมูลที่ต้องการจากตารางหรือ View เฉพาะ นำผลลัพธ์เข้าสู่โครงสร้างข้อมูล
Python (เช่น DataFrame)

14. การเก็บรวบรวมข้อมูลจากหน้าเว็บ: สำหรับเว็บไซต์ที่มีโครงสร้างหรือกึ่งโครงสร้างใช้ไลบรารี Python (เช่น BeautifulSoup) เพื่อแยกวิเคราะห์เนื้อหา HTML/XML และดึงข้อมูลที่ต้องการโดยใช้แท็กหรือ CSS Selector หากเว็บไซต์ไม่มีโครงสร้างหรือต้องการการโต้ตอบให้พิจารณาใช้เครื่องมืออย่าง Selenium

สิ่งสำคัญคือต้องรักษาการสื่อสารที่ชัดเจนกับลูกค้าตลอดกระบวนการบันทึกความคืบหน้าอย่างสม่ำเสมอ และติดตามความท้าทายหรือวิธีแก้ปัญหาที่นำมาใช้

2.3.1 แบบฝึกหัด – การบูรณาการข้อมูล

2.3.1.1 การตั้งค่า

```
import numpy as np
import pandas as pd

df = pd.read_csv('/content/sample_data/california_housing_test.csv')
df.head()
```

ชุดข้อมูลนี้ประกอบด้วยข้อมูลที่อยู่อาศัยในรัฐแคลิฟอร์เนีย มีทั้งหมด 3,000 รายการ และ 9 คอลัมน์ ได้แก่ longitude, latitude, housing_median_age, total_rooms, total_bedrooms, population, households, median_income และ median_house_value

2.3.1.2 การต่อข้อมูล (Concatenation)

เอกสารอ้างอิง: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.concat.html>

การต่อข้อมูลแนวแถว (axis=0): นำ DataFrame หลายชุดมาเรียงต่อกันในแนวแถว ผลลัพธ์คือ DataFrame ที่มีจำนวนแถวเพิ่มขึ้น แต่คอลัมน์เดิม

```
df_1 = df[['longitude', 'latitude', 'median_income']].sample(n=5)
```



```
df_2 = df[['longitude', 'latitude', 'median_income']].sample(n=5)
df_3 = df[['longitude', 'latitude', 'median_income']].sample(n=5)
df_cat1 = pd.concat([df_1, df_2, df_3], axis=0)
```

การต่อข้อมูลแนวคอลัมน์ (axis=1): นำ DataFrame หลายชุดมาเรียงต่อกันในแนวคอลัมน์ ผลลัพธ์คือ DataFrame ที่มีจำนวนคอลัมน์เพิ่มขึ้น หาก Index ไม่ตรงกันจะแสดงค่า NaN

```
df_1 = df[['longitude']][:5]
df_2 = df[['latitude']][:5]
df_3 = df[['median_income']][:5]
df_cat2 = pd.concat([df_1, df_2, df_3], axis=1)
```

2.3.1.3 การรวมข้อมูล (Merging)

เอกสารอ้างอิง : <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html>

การรวมข้อมูลแบบ Merge ใช้สำหรับรวม DataFrame โดยอ้างอิงคอลัมน์ร่วม (Key Column) รองรับหลายรูปแบบ ได้แก่:

15. Inner Join: รวมเฉพาะแถวที่มีค่า Key ตรงกันในทั้งสอง DataFrame
16. Outer Join: รวมทุกแถวจากทั้งสอง DataFrame และเติม NaN ในส่วนที่ไม่มีข้อมูลตรงกัน
17. Left Join: รวมทุกแถวจาก DataFrame ซ้าย และเฉพาะแถวที่ตรงกันจาก DataFrame ขวา
18. Right Join: รวมทุกแถวจาก DataFrame ขวา และเฉพาะแถวที่ตรงกันจาก DataFrame ซ้าย

```
df_1 = df[['longitude', 'median_income']][0:5]
df_2 = df[['longitude', 'median_house_value']][0:5]
```

```
pd.merge(df_1, df_2, on=['longitude'],
         how='inner')

pd.merge(df_1, df_3, on='longitude',
         how='outer').drop_duplicates()
```

2.3.1.4 การเชื่อมต่อข้อมูล (Joining)

เ อ ก ส า ร อ ั า ง อ ี ง :

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.join.html>

การ Join ใช้สำหรับเชื่อมต่อ DataFrame โดยอ้างอิง Index ของข้อมูล รองรับทั้ง left, right, inner และ outer join ซึ่งมีผลลัพธ์คล้ายกับ Merge แต่ใช้ Index แทนคอลัมน์เป็นตัวอ้างอิง

```
df_1 = df[['longitude']][0:5]
df_2 = df[['latitude']][2:7]
df_1.join(df_2, how='left') # Left Join
df_1.join(df_2, how='right') # Right Join
df_1.join(df_2, how='inner') # Inner Join
df_1.join(df_2, how='outer') # Outer Join
```

2.3.2 กรณีศึกษา – เงินเดือนนักวิทยาศาสตร์ข้อมูล

2.3.2.1 การตั้งค่า

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('/content/Data Science Jobs
Salaries.csv', skiprows=2)
```

ชุดข้อมูลนี้ประกอบด้วยข้อมูลเงินเดือนของงานสาขาวิทยาศาสตร์ข้อมูล มีทั้งหมด 245 รายการ และ 11 คอลัมน์ ได้แก่ work_year, experience_level, employment_type, job_title, salary, salary_currency, salary_in_usd, employee_residence, remote_ratio, company_location และ company_size

2.3.2.2 การต่อข้อมูล (Concatenation)

ทดลองต่อข้อมูลเงินเดือนนักวิทยาศาสตร์ข้อมูลจาก 3 กลุ่มตัวอย่าง แต่ละกลุ่มมี 5 รายการ โดยเลือกคอลัมน์ `company_location`, `job_title`, `experience_level` และ `salary_in_usd`:

```
df_1 = df[['company_location', 'job_title', 'experience_level', 'salary_in_usd']].sample(n=5)
df_2 = df[['company_location', 'job_title', 'experience_level', 'salary_in_usd']].sample(n=5)
df_3 = df[['company_location', 'job_title', 'experience_level', 'salary_in_usd']].sample(n=5)
df_cat1 = pd.concat([df_1, df_2, df_3], axis=0)
# ต่อแนวแถว
df_cat2 = pd.concat([df_1, df_2, df_3], axis=1)
# ต่อแนวคอลัมน์
```

2.3.2.3 การรวมข้อมูล (Merging)

ทดลองรวมข้อมูลเงินเดือนโดยใช้ `company_location` เป็น Key Column:

```
df_1 = df[['company_location', 'experience_level', 'salary_in_usd']][0:5]
df_2 = df[['company_location', 'job_title', 'salary_in_usd']][0:5]
pd.merge(df_1, df_2, on='company_location', how='inner')
pd.merge(df_1, df_2, on='company_location', how='inner').drop_duplicates()
```

สังเกตว่าการ Merge ด้วย `company_location` อาจทำให้เกิดแถวซ้ำ หากมีหลาย Job Title ในตำแหน่งเดียวกัน ใช้ `drop_duplicates()` เพื่อลบแถวที่ซ้ำกันออก

```
pd.merge(df_1, df_3, on='company_location',  
how='outer').drop_duplicates()
```

2.3.2.4 การเชื่อมต่อข้อมูล (Joining)

ทดลอง Join ข้อมูลเงินเดือนโดยใช้ Index:

```
df_1 = df[['experience_level']][0:5]  
df_2 = df[['job_title']][2:7]  
df_1.join(df_2, how='left').drop_duplicates()  
df_1.join(df_2, how='right').drop_duplicates()  
df_1.join(df_2, how='inner').drop_duplicates()  
df_1.join(df_2, how='outer').drop_duplicates()
```